

PROGRAMMING INTERACTIVE SYSTEM (CSC541)



Instructor Name :- Dr. Suzana Binti Ahmad

Room No :- Tb-01

E-mail :- suzana@tmsk.uitm.edu.my

syllabus

Topics

Introduction to Human Computer Interaction

User Interface Programming

Peripherals

Software Architectures and MVC

Psychology and Motor Performance

Cognition Aspects

UCD - User Centered Design

Web Development

Emergent Interfaces and Interactions



- **Assessment 1 (20%)**
 - Test
- **Assessment 2 (20%)**
 - Lab Assignments
- **Assessment 3 (30%)**
 - Project (paper prototype & System prototype)
- **Assessment 4 (30%)**
 - Final Test

“What are intelligent machines for if not interacting with people”

Jonathan Grudin



Unplug



- What interactive systems do you use in your day-to-day life?

Interactive Systems -



- Until 1980s almost all commercial computer systems were non-interactive
- Computer operators would set-up the machine to read in a large volumes of data
- Computer would then process each input and generate appropriate output

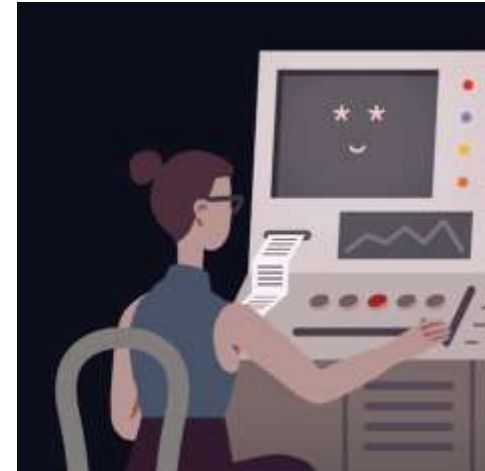
Interactive Systems -

- Most systems are interactive (involve users directly)
- Interactive systems – user and computer exchange information frequently and dynamically.



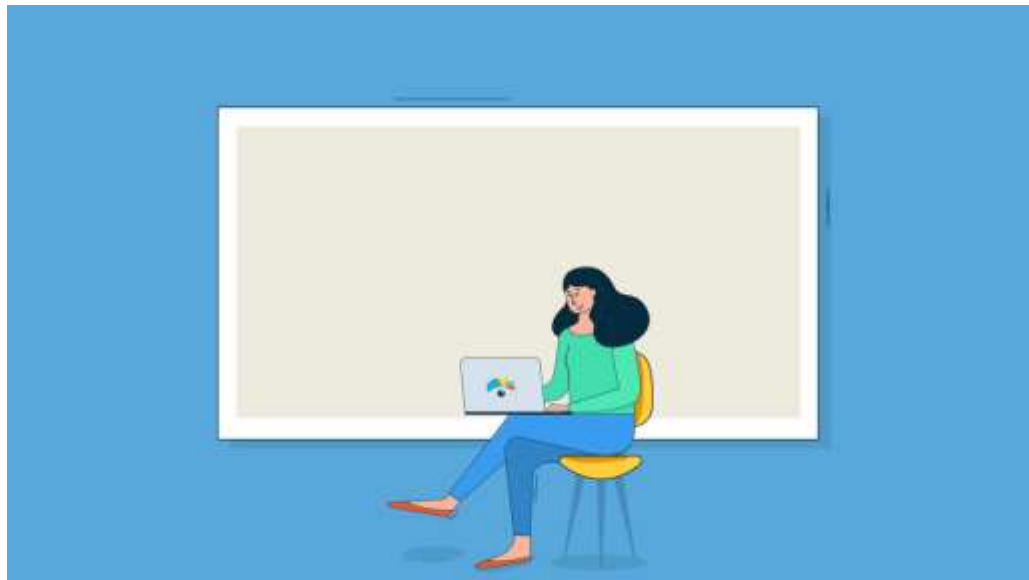
Understanding – Norman's evaluation model

1. User has a goal
2. User looks at system and attempts to work out how he/she would execute a series of tasks to achieve the goal
3. User carries out some actions
4. System responds to the actions and presents results to the user
5. User looks at the results of his action and attempts to evaluate whether or not the goals have been achieved

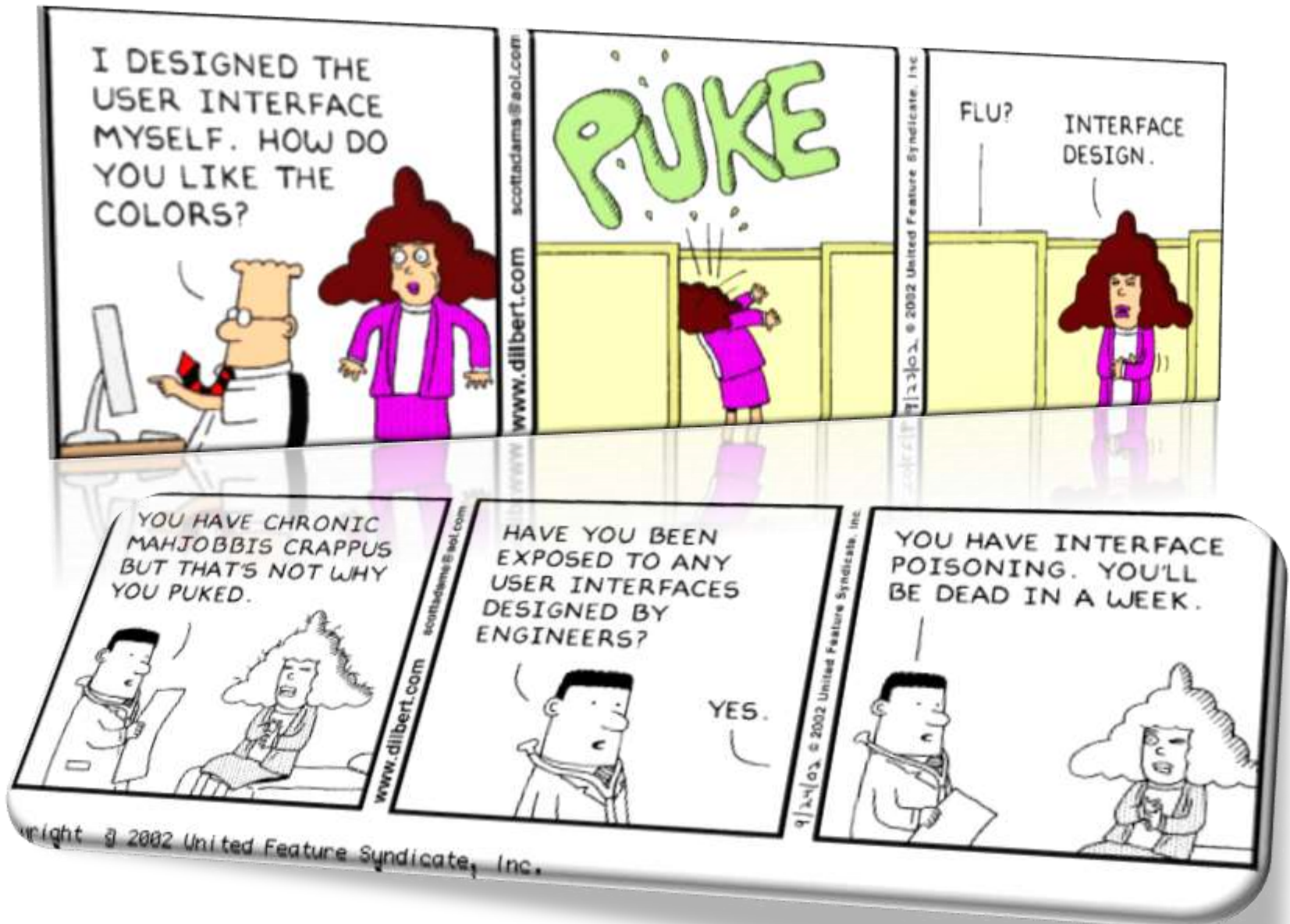


A good interactive system

- User can easily work out how to operate the system in an attempt to achieve his goals
- User can easily evaluate the results of his action on the system



Why Bother?



Why Systems Fail?

- Inadequate requirements 13%
- Lack of user participation 12%
- Inadequate resources 11%
- Unrealistic expectations 10%
- Lack of support at senior level 9%
- Changing specification 8%
- Lack of planning 8%

The Perfect User

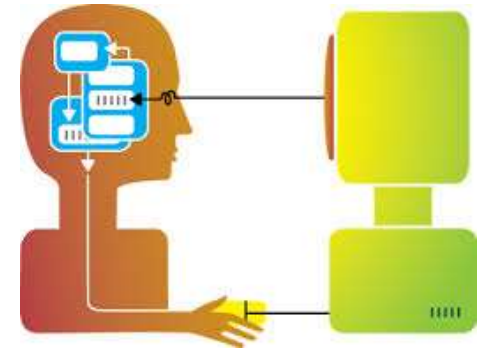
(every designer's wish)



Common Issues in User Interface Design

- Software developers are forced to “do it all”
- Often based on intuition and experience than on theory-based models
- Tendency to let the art of interface design beats its usability
- Inconsistent features that do not fit into a good user interface design criteria

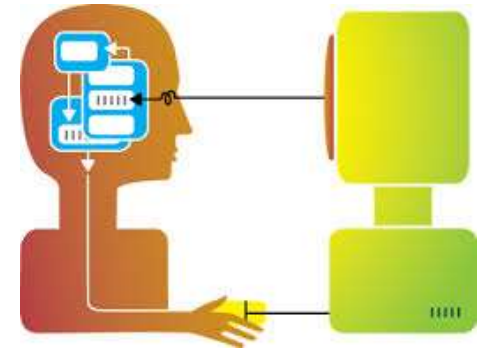
Why Study HCI?



Business view :

- to **employ** people more productively and effectively
 - people costs now far outweigh hardware and software costs
- people now **expect** “easy to use” systems
 - generally they are not tolerant of poorly designed systems
 - if a product is hard to use, they will seek other products

Why Study HCI?



Human Factors view :

- Humans have limitations.
- Errors are costly in terms of
 - loss of time & money
 - loss of lives in critical systems
 - loss of morale

What is HCI

Short for *human-computer Interaction*.

A discipline concerned with the study, design, evaluation and implementation of interactive computing systems for human use and with the study of major phenomena surrounding them



What is HCI

Design of interactive systems that are:

- Enjoyable to use – useful – enhance the lives of the people that use them
- Accessible
- Usable
- Engaging



What is HCI

Methods for:

- Capturing what people want to do rather than just what the technology/designer can do
- Understanding how to translate from what people want to good design
- Involving people in the design process
- Designing for diverse users and uses



What is HCI

- The Human
 - Single user, groups, I/O channels, memory, reasoning, problem solving, error, psychology
- The Computer
 - Desktop, embedded system, data entry devices, output devices, memory, processing
- The Interaction
 - Direct/indirect communication, models, frameworks, styles, ergonomics

Video

- https://www.youtube.com/watch?v=J_FvdyhtVHg

History of HCI

- Douglas Engelbart, 1962
“Augmenting Human Intellect: A
Conceptual Framework”
- In 1968, workstation with a mouse,
links across documents, chorded
keyboard



History of HCI

- XEROX Alto and Star
 - Windows
 - Menus
 - Scrollbars
 - Pointing
 - Consistency
- Apple LISA and Mac
 - Inexpensive
 - High-quality graphics
 - 3rd party applications

History (and future) of HCI

- Large displays
- Small displays
- Peripheral displays
- Alternative I/O
- Ubiquitous computing
- Virtual environments
- Implants
- Speech recognition
- Multimedia
- Video conferencing
- Artificial intelligence
- Software agents
- Recommender systems
- ...



The goal of HCI

The goals of HCI are to develop or improve the safety, utility and effectiveness of systems that include computers, often through improving **usability**.

The goal of UI Design



- 5 (plus 1) measurable factors
 - Time to learn
 - Speed of performance
 - Error rate
 - Retention over time
 - Subjective satisfaction
 - Cost

Unplug Activity 2

- Let us look at some human designs
- What do you think?
- Why?









Things to Ponder

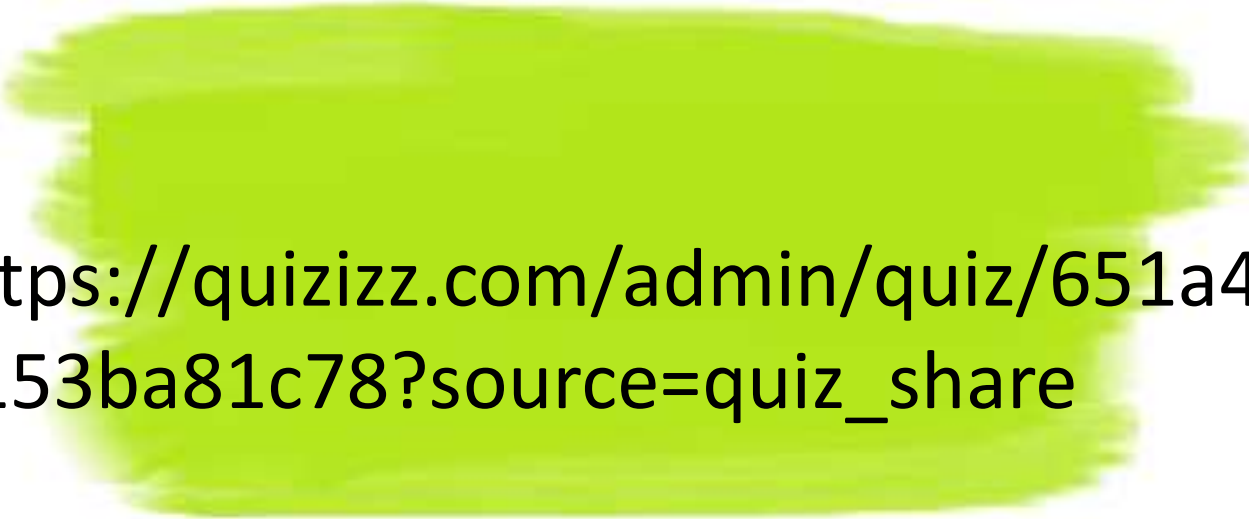
What do humans do well?

- Sense low level stimuli
- Pattern recognition
- Inductive reasoning
- Multiple strategies
- Adapting
- “Hard and fuzzy things”, paraphrasing George Miller

What do computers do well?

- Counting and measuring
- Accurate storage and recall
- Rapid and consistent responses
- Data processing/calculation
- Repetitive actions
- “Simple and sharply defined things”
paraphrasing George Miller

Let us recap



https://quizizz.com/admin/quiz/651a40acd805c3153ba81c78?source=quiz_share

End of Lecture

USER INTERFACE PROGRAMMING



WEEK TWO

Instructor Name :- Dr. Suzana Binti Ahmad

Interface

=

user interface

Interactive system

=

human-computer interactive system

? Interface

- An interface is a set of commands or menus through which a user communicates with a program



Interface

=

user interface

Interactive system

=

human-computer interactive system

A user interface is well-designed when the program behaves exactly how the user thought it would

* lets watch video

<https://www.youtube.com/watch?v=qLI0vlchsls>



User Interface

- User interface is one of the most important parts of any program because it determines how easily user can make the program do what they want to do.
- UI is important because it affects
 - The feelings
 - The emotions
 - MoodOf your users



User Interface



- UI determines how easily user can make the program do what they want to do.
- A powerful program with a poorly designed user interface has a little value.
- If the UI is wrong and the user feels like they can't control your software, they literally won't be happy and they'll blame it on your software
- If they UI is smart and things work the way the user expected them to work, they will be cheerful to use the software

Types of User Interface

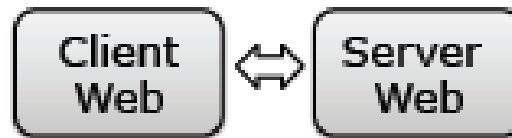
1. **Command language** – user must know the machine and program-specific instructions or codes
2. **Menus** – user chooses the commands from lists displayed on the screen
3. **Graphical user interface (GUI)** – user gives commands by selecting and clicking on icons displayed on the screen

Interface
=
user interface

Interactive system
=
human-computer interactive system

Interactive system

- Software that interacts with its environment





What is Interactive System ?

- Any system that accepts input from the user and provides information as output to the user.
- An interactive system virtually includes all types of devices and systems which a human interacts with
- It talks about a few types of interaction in computer systems



Interactive System

- Types of interaction :
 - Interaction between Human and Computer (Human-Computer Interaction)
 - Interaction between computer systems
 - Interaction between users through the computer system



Interactive System

- Interactive systems can be viewed from:
 - The user perspective
 - The system perspective



Interactive System

- To the user the system is:
 - A tool for accomplishing tasks
 - Something that receives input
 - And hopefully, displays output
 - All too often, a source of frustration



Interactive System

- To the system the user is:
 - A source of a synchronous input
 - An indirect target for output through various devices (display, speaker, lights, ...)

The main purpose of UI programming
is to produce an

INTERACTIVE SYSTEM



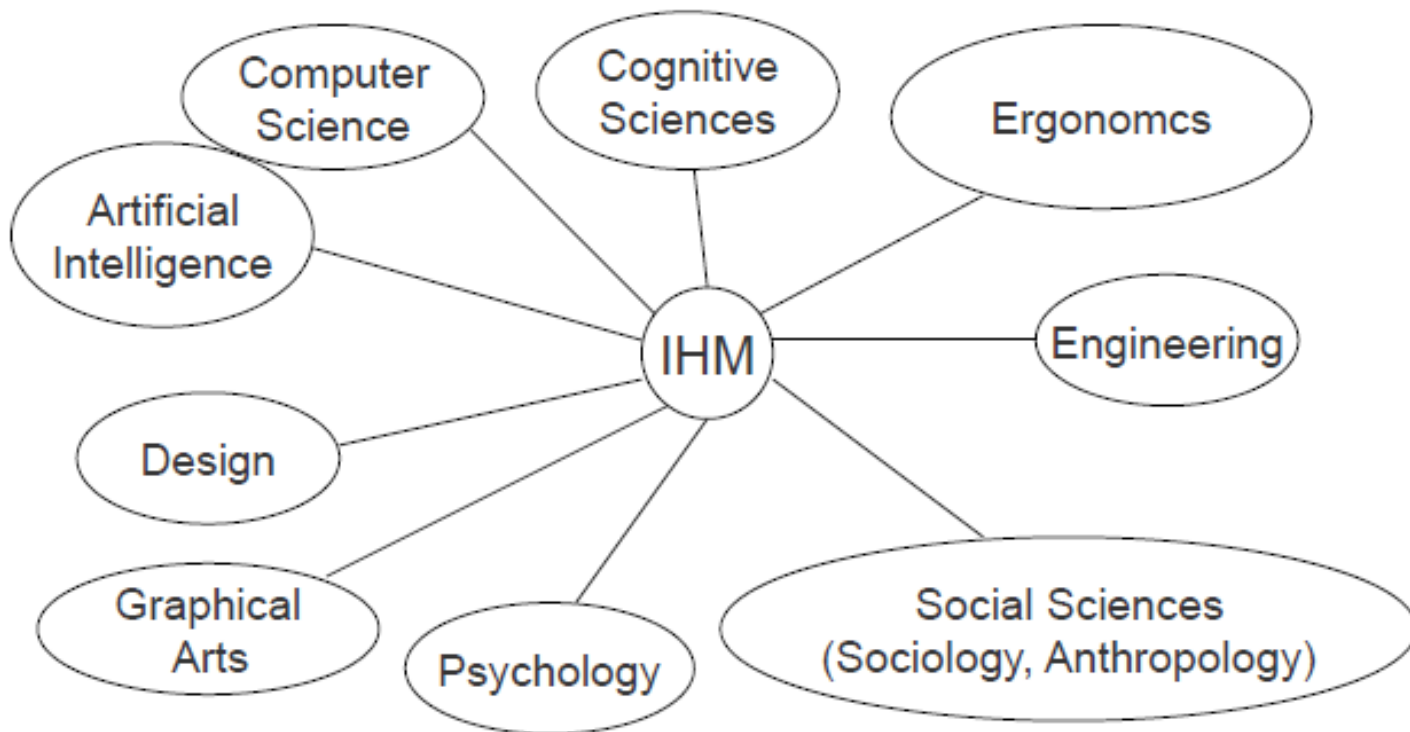
Engineers and computer scientists are not (by default) good interface designers:

they (we) are expert computer users,
their (our) interest is the computer or the
interface

What interests users is what the interface and the
computer helps them do

We have to design **FOR** and **WITH** users

HCI is multidisciplinary



importance

- The quality of UI is important as they are used worldwide and in every domain, including critical systems (where lives are at risk)

Interface
=
user interface

Interactive system
=
human-computer interactive system

Human-computer interactive system

- Interactive system whose environment is one or more (human) users



* let's watch video

<https://www.youtube.com/watch?v=JCtrNLt1wgl>

Human Computer Interaction

“HCI is a discipline concerned with the **design**,
evaluation and **implementation** of
interactive computing systems for human use
and with the study of major **phenomena**
surrounding them” ACM definition

Importance - example

- Machine for paying parking fees at Tullamarine airport in Melbourne
- For a year, an airport employee was hired to help users





What is PROGRAMMING

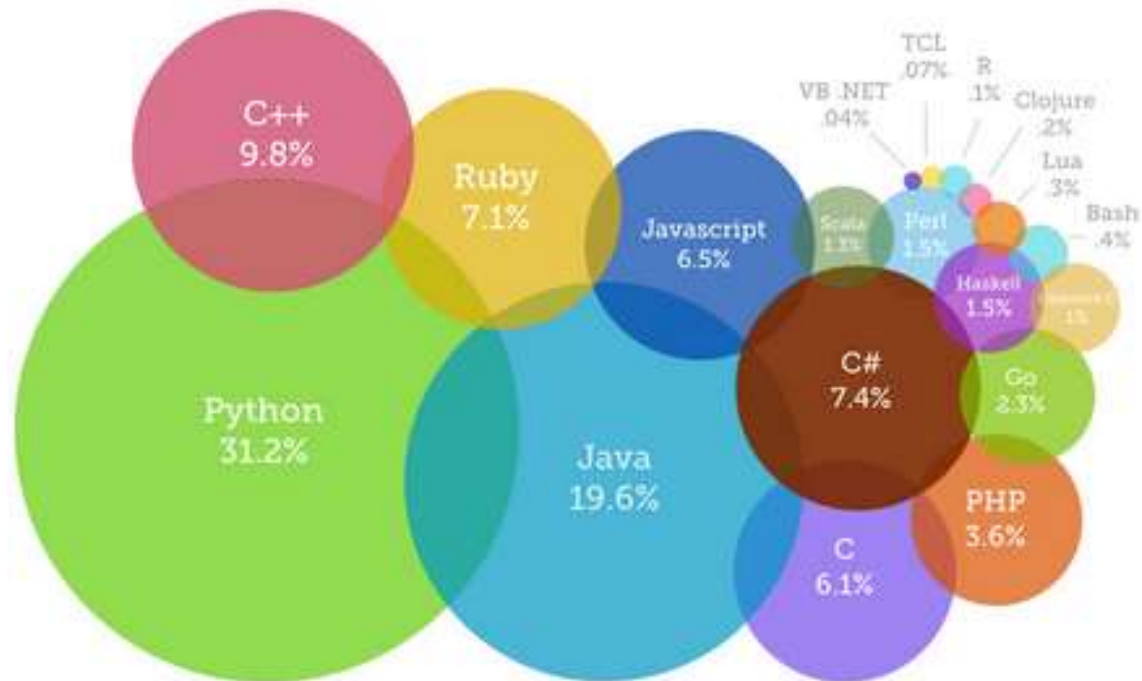
Programming



- Programming is a creative process done by programmers to instruct a computer on how to do a tasks.
- Computers can be programmed to do interesting things
- Programming is to
 - Discover a set of rules with real world effects.
 - Design idealized representations of these rules and the real world objects affected by these rules.
 - Understand the interaction of these rules and their objects.
 - Optimize the interactions of these rules for the constraints of safety, budget, correctness, and other resources.

Programming Languages

Most Popular Coding Languages of 2015



UI Tools and Programming Languages

- UI software is often large, complex and difficult to implement, debug and modify.
- As the interfaces become easier to use, they become harder to create [Myers 1994].



UI Tools and Programming Languages

- Fortunately, there has been significant progress in software tools to help with creating interfaces
- There have been special software systems and tools to help design and implement the user interface software.



Components of User Interface Software

- User interface software may be divided into various layers such as:
 - Windowing system
 - Toolkit
 - Higher-level tools

Graphical User Interface (GUI)

- Interface (that uses output peripherals (screen, projector))



- Some input peripherals (mouse, pen) that provide relative position w.r.t the output peripherals



- Allow reference to aspects on the interface using pointing (thus linking input/output)
- let's watch video
- <https://www.youtube.com/watch?v=MF78pN3iN5s>

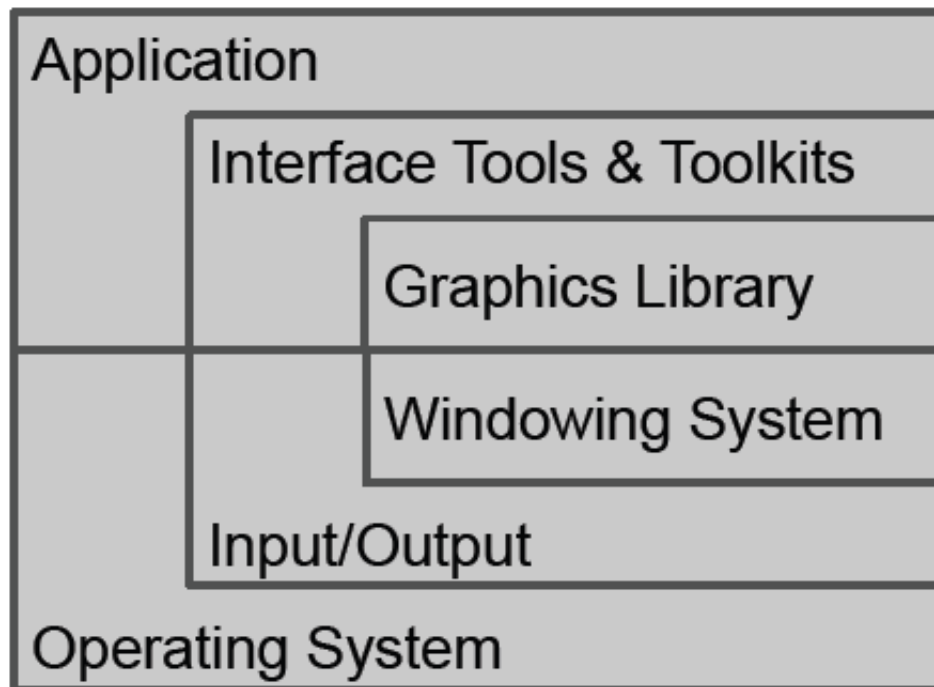
Lets define SOFTWARE

What is Software

- Set of machine-readable instructions that directs a computer's processor to perform specific operations
- Software can be divided into
 - System / platform software
 - Application software
 - User-written software



Software layers



Applications/Communication (MacApp)

Builders,
Java Swing, Qt, GTK+, MFC, Cocoa

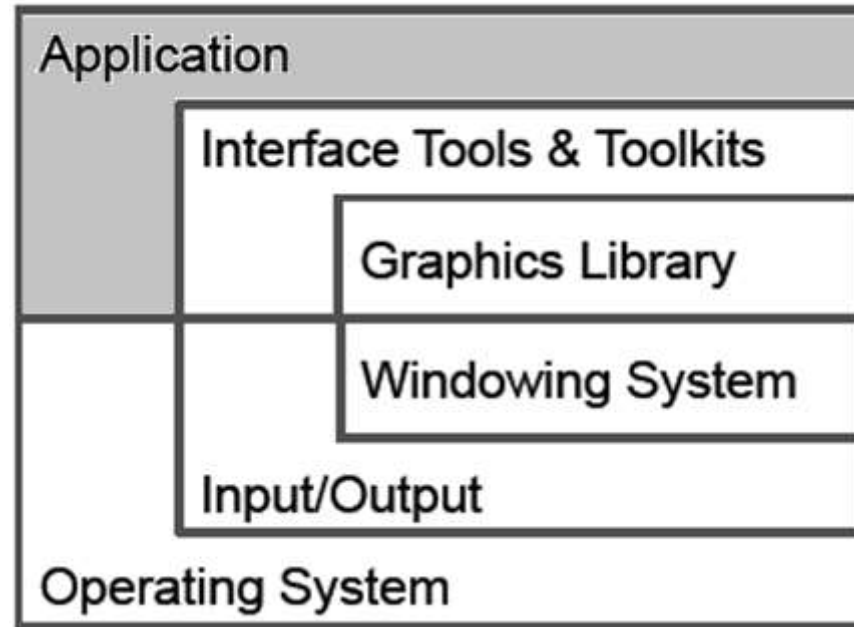
GDI+, Quartz, GTK+/Xlib, OpenGL

X Windows (+KDE or GNU)



Windows, Mac OS, Unix, Linux,
Android, iOS, WindowsCE

Application layers



Application Software



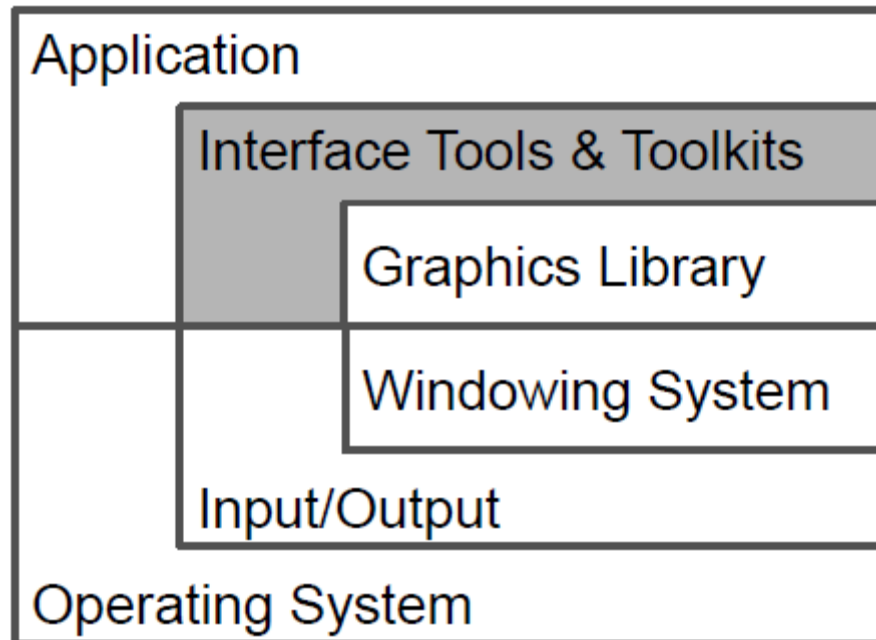
- Application software or Applications are what most people think of when they think of software
- Typical examples include office suites and video games
- Application software is often purchased separately from computer hardware
- Sometimes applications are bundled with the computer, but that does not change the fact that they run as independent applications

Application Software

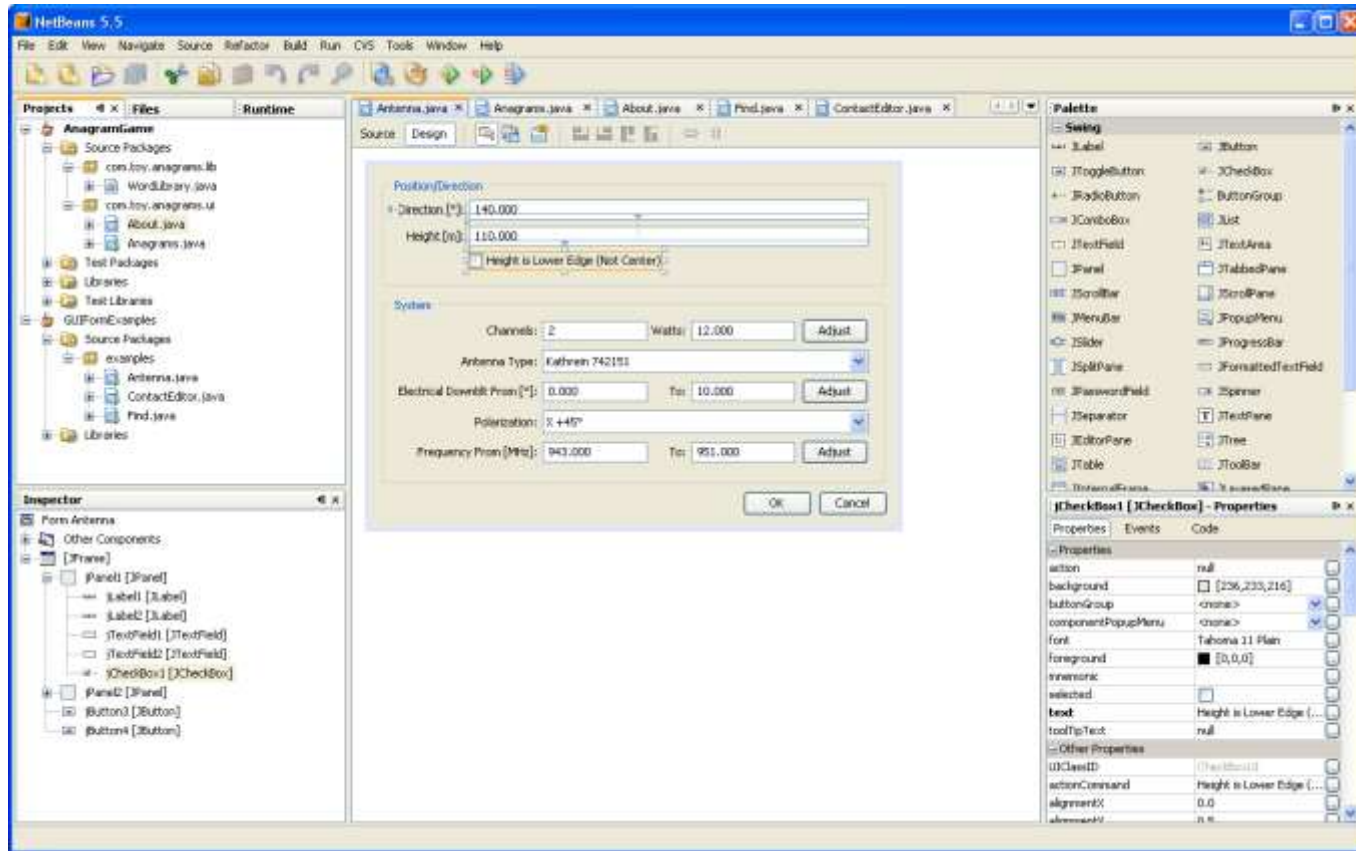


- Application are usually independent programs from the operating system, though they are often tailored for specific platforms
- Most users think of compilers, databases, and other 'system software' as applications which is not.

Software layers



Interface builders



Examples : MS Visual Studio (C++, C#, etc.),
NetBeans (Java), Interface Builder (ObjectiveC),
Android Layout Editor

Interface builders

Can be used to

- Create prototypes (but attention it looks real)
- Get the << look >> right
- Be part of final product
- Design is fast
- Modest technical training needed
- Can write user manuals from it

But: still need to program (and clean code...)

Interface toolkits

Libraries of interactive object (<<widgets>>, e.g. buttons) that we use to construct interfaces

Functions to help programming of GUIs

Also used to handle input events

Interface toolkits

Toolkit	Platform	Language
Qt	multiplatform	C++
GTK+	multiplatform	C
MFC later WTL	Windows	C++
WPF (subset of WTL)	Windows	(any .Net language)
FLTK	multiplatform	C++
AWT / Swing	multiplatform	Java
Cocoa	MacOs	Objective C
Gnustep	Linux, Windows	Objective C
Motif	Linux	C
jQuery UI	Web	javascript

Toolkit



- Library of widgets (also called controls)
 - Eg – menus, buttons, scroll bars, and text input fields
- Creating an interface using a toolkit can only be done by programmers, because only have a procedural interface
- Technique known as **Event based programming**
- Adv. – final UI will look and act similarly to other UIs created by the same toolkit. (does not need to rewrite the std functions)
- DisAdv –the styles of interaction are limited to those provided.

Toolkit

– Example of available toolkits

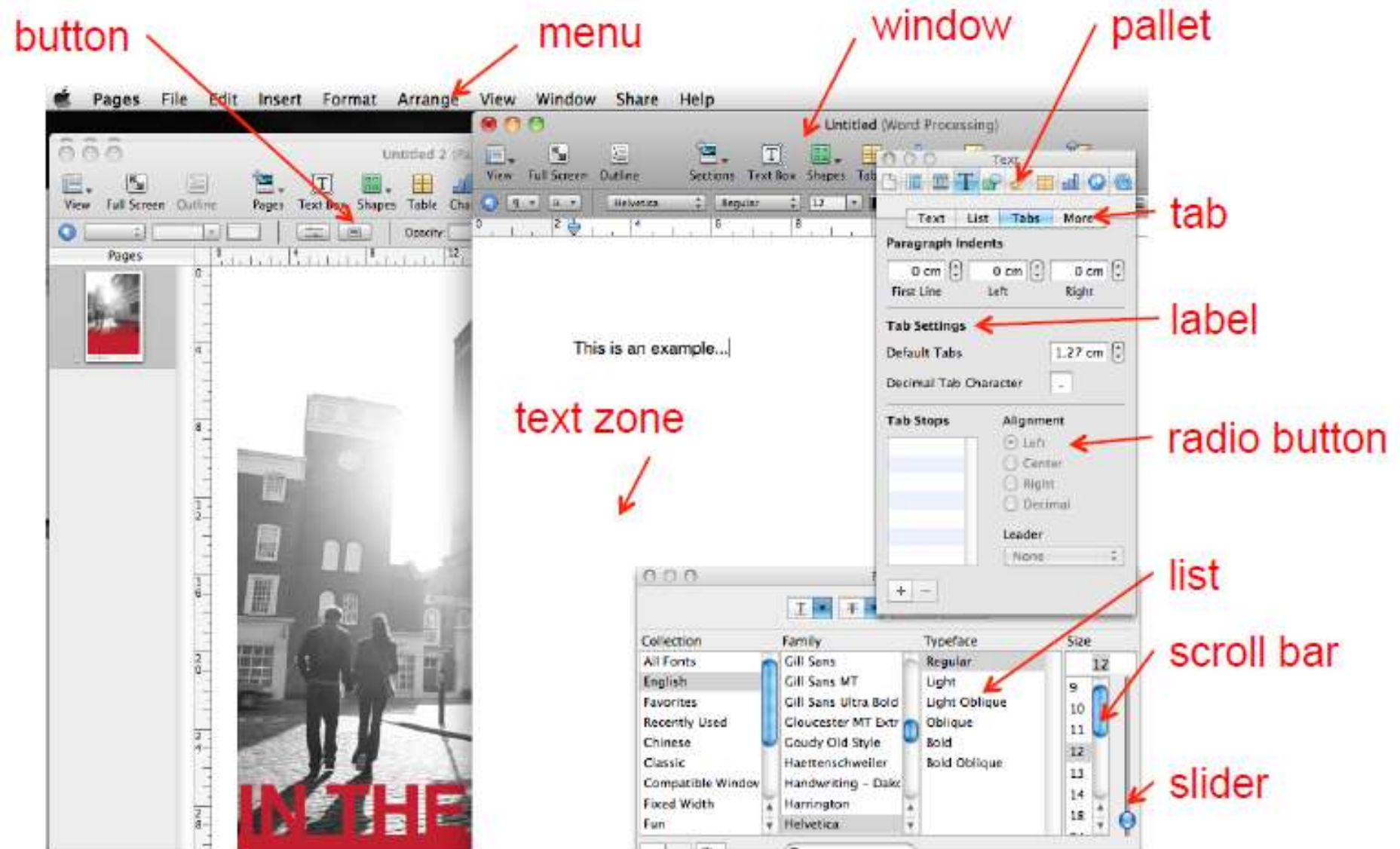
- Smalltalk
- Macintosh toolbox
- Java Swing toolkit
- Motif
- InterViews
- Amulet
- SUIT



?? Widgets



<< widgets >> (window gadget)



Window gadget (widgets)



- Widget is a generic term for the part of a GUI that allows the user to interface with the application and operating system
- Widgets display information and invite the user to act in a number of ways
- Typical widgets include buttons, dialog boxes, pop-up windows, pull-down menus, icons, scroll bars, resizable window edges, progress indicators, selection boxes, windows, tear-off menus, menu bars, toggle switches and forms

Window gadget (widgets)



- The term also refers to the program that is written in order to make the graphic widget in the GUI look and perform in a specified way, depending on what action the user takes while interfacing with the GUI
- The term widget is used to refer to either the graphic component or its controlling program or to refer to the combination of both

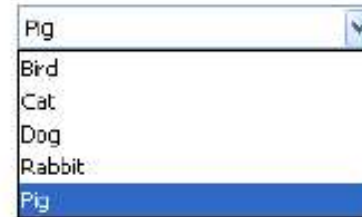
Swing widgets



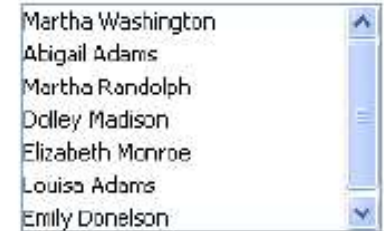
[JButton](#)



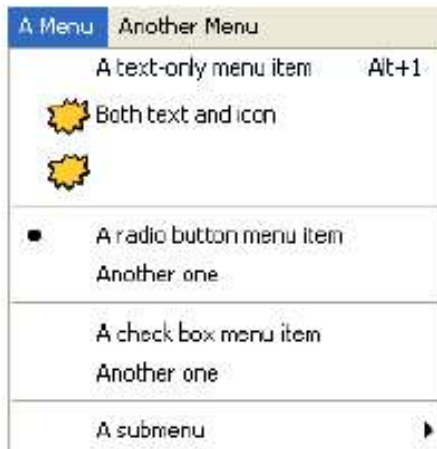
[JCheckBox](#)



[JComboBox](#)



[JList](#)



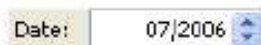
[JMenu](#)



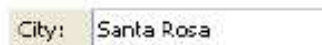
[JRadioButton](#)



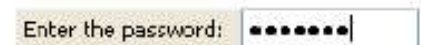
[JSlider](#)



[JSpinner](#)



[JTextField](#)



[JPasswordField](#)

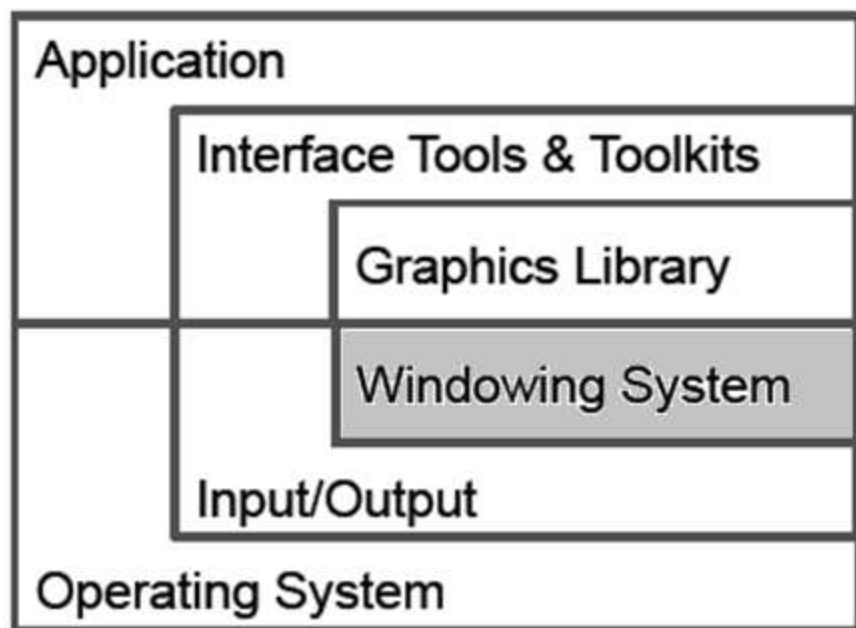
Widget complexity

- Simple widgets
 - Buttons, scroll bars, labels, ...
- Composite / complex widgets
 - Contain other widgets (simple or complex)
 - Dialog boxes, menus, color pickers, ...

Benefit of a widget

- Presentation
 - appearance
- Behavior
 - Reaction to user actions

Interface with the application: notification of state changes



Windowing system



- Supports the separation of the screen into different (usually rectangular) regions, called windows
- [scheifler 1986] divides window functionality into 2 layers:
 - **Window system** – the functional or programming interface
 - **Window manager**- user interface

Windowing system



– ***Window system***

- Provides procedures that allow the application to draw pictures on the screen and get input from the user

– ***Window manager***

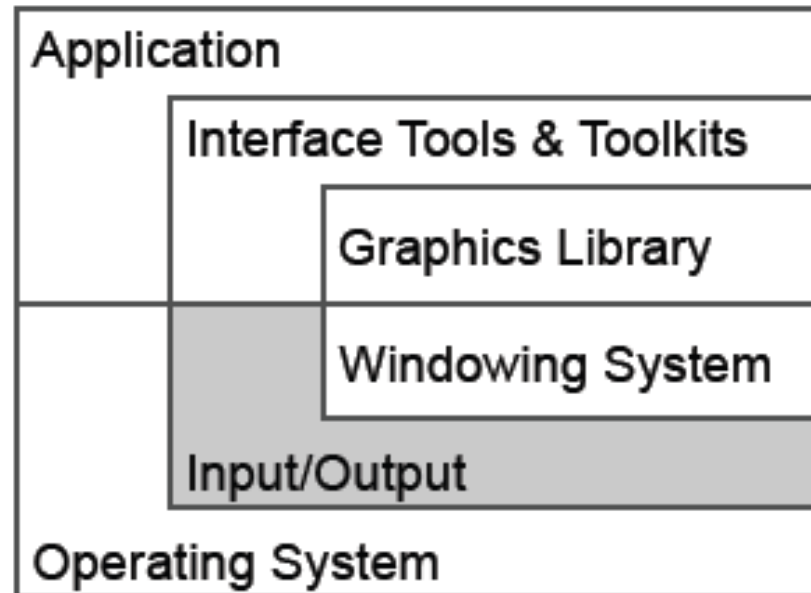
- Allows the end user to move windows around and is responsible for displaying the title lines, borders, and icons around the windows

Windowing system



- As the programmers need to use the application-specific parts of the interface, window system interface must be used directly
- Therefore, the windowing system's programming interface has significant impact on most user interface programmers

Input / Output layer



Input / output peripherals

- Input : where we give commands



- Output : where the system shows its state



Interactivity vs. computing

- Closed systems (computation) :
 - Read input, compute, produce result
 - Final state (end of computation)
- Open systems (interaction):
 - Events/changes caused by environment
 - Infinite loop, non-deterministic
- That relates with 2 programming paradigm
 - Let's watch video
 - <https://www.coursera.org/lecture/object-oriented-java/core-introduction-to-event-driven-programming-j3FBx>

Let's talk more on Event base Programming



EVENT BASED
PROGRAMMING

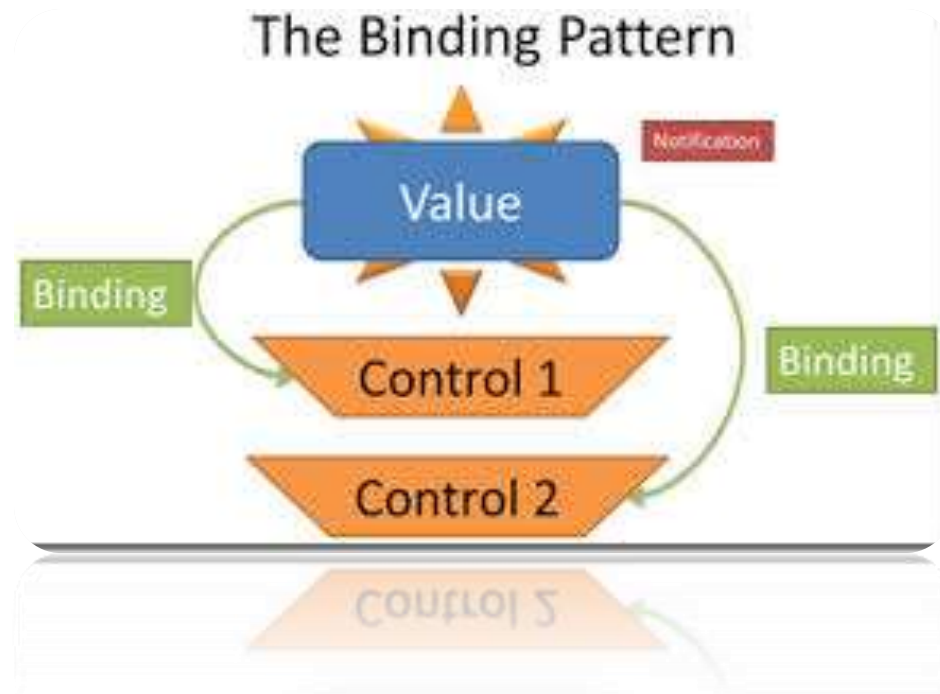
Event based Programming

- In computer programming, event-driven programming is a programming paradigm in which the flow of the program is determined by events such as user actions
 - Mouse clicks, key pressed, sensor outputs,
- Event-driven programming is the dominant paradigm used in graphical user interfaces and other applications



Event based Programming

- In an event-driven application, there is generally a main loop that listens for events, and then triggers a callback function when one of those events is detected





Event based Programming

- In embedded systems the same may be achieved using hardware interrupts instead of a constantly running main loop.
- Event-driven programs can be written in any programming language, although the task is easier in languages that provide high-level abstraction, such as closures.



Event based Programming

- The first step in developing event-driven program is to :
 - Write a series of subroutines, or methods, called event-handler routines
 - These routines handle the events to which the main program will respond
 - For example, a single left-button mouse-click on a common button in a GUI program may trigger a routine that will open another window, save data to a database or exit the application



Event based Programming

- Many modern day programming environments provide the programmer with event templates so that the programmer only needs to supply the event code



Event based Programming

- The second step is to :
 - bind event handlers to events so that the correct function is called when the event takes place
 - Graphical editors combine the first two steps double-click on a button, and the editor creates an (empty) event handler associated with the user clicking the button and opens a text window so you can edit the event handler.



Event based Programming

- The third step is to :
 - Write the main loop
 - This is a function that checks for the occurrence of events, and then calls the matching event handler to process it
 - Most event-driven programming environments are already being provided. Therefore, it need not to be specifically provided by the application programmer.

Higher Level Tools

- Programming at the toolkit level is quite difficult, there is a tremendous interest in higher level tools – makes the user interface software production process easier

Higher Level Tools

- Classification of higher level tools are
 - Language based
 - State Transition Networks
 - Context-Free Grammars
 - Event Languages
 - Declarative Languages
 - Constraint Languages
 - Database Interface
 - Visual Programming
 - Object based

End of Lecture